RETAIL DATA AGENT · BRAND SCOPING · AS OF 2026-10-31

Why does a brand manager see a smaller item count on an order?

One order can hold items from several brands. The brand filter lets a Levi's manager see that order, because it has a Levi's item, but the order's own `num_of_item` still counted every brand's items. So we recount it: a brand manager sees how many of *their* items are in the order. See also how brand scoping works.

🟩 The user's own brand ⬚ Another brand 🟥 The leak

In short

1. The brand filter picks *which* orders a manager sees, but it didn't change the numbers *inside* each order.
2. `num_of_item` was the one such number: it counted other brands' items too, so it gave away other brands' sales.
3. Now, for a brand manager, it counts only their brands' items. The CEO still sees the real total.

One basket, seen by the Levi's manager

ORDER 323 · REAL DATA

Levi's	Other brand	Other brand	Other brand
1 item	1 item	1 item	1 item

BEFORE THE FIX

4

The manager learns that this customer bought 3 items from other brands, which isn't their data.

AFTER THE FIX

1

The manager sees their own item only. The other brands' items don't exist for them.

Checked on real orders

Three real orders that mix Levi's with other brands, checked on BigQuery on 2026-10-31:

ORDER	ALL ITEMS	LEVI'S	MANAGER SEES
323	4	1	1
400	4	1	1
452	2	1	1

That query scanned 4.5 MB and took 1.7 seconds.

How the recount works

The filtered `orders` table keeps every column as it is, and swaps only `num_of_item` for a count taken from the filtered items, which are already the user's brands only:

```sql
-- every column, but replace num_of_item with
-- the number of the user's items in this order
SELECT * REPLACE (
  (SELECT COUNT(*)
   FROM _brand_scoped_order_items AS items
   WHERE items.order_id = orders.order_id)
  AS num_of_item)
FROM `…orders` AS orders
-- only orders with at least one of the user's items
WHERE order_id IN (
  SELECT order_id FROM _brand_scoped_order_items)
```

The AI's SQL doesn't change. Whether it writes `num_of_item`, `SUM(num_of_item)` or `AVG(num_of_item)`, it gets the recounted number. The column's note in `schema.yaml` tells the AI what the number means for a brand manager.

Recount, or block the column?

RECOUNT · CHOSEN

The number stays useful ("how many of my items per order") and every query shape is covered.

BLOCK THE COLUMN · DROPPED

Reject any query that uses `num_of_item`. But `SELECT *` uses it without naming it, so we'd have to track columns through every query, which is harder to get right and loses a useful number.

Since ticket 08, the AI's `SELECT *` is rejected and every column is traced by `query_safety/step2_personal_data/columns.py`, so blocking the column would now be easy. The recount stays because the number is still useful. See the parser rules.

The other `orders` columns (status and dates) describe the order itself, not other brands' items, so they need no change.

The rule in one sentence

A brand filter must cover the numbers inside a row as well as the rows themselves, so any count that includes other brands is recounted from the user's own items.

Code: `orders_columns_with_scoped_item_count` in `src/retail_agent/query_safety/step3_brand_scope/order_item_count.py` builds the `SELECT * REPLACE (…)` list. `BrandScope` in `src/retail_agent/query_safety/step3_brand_scope/brand_scope.py` uses it for the filtered `orders` table (`_COLUMNS`), next to the row filter in `_FILTERS`. The CEO's queries aren't rewritten, so they keep the real count.

Tests: `tests/test_step3_order_item_count.py`, and the exact rewritten SQL in `tests/test_step3_brand_scope.py`.

See also brand scoping, architecture.